

Final Year Design Project System Requirements Specification

IoT-Based Smart Home Automation System Using ESP32

Software Design Specification Document

by

M. Mohsin Saleem	BCS22006
Abdul Rehman Tahir	BCS22016
Rehan Ali	BCS22021
Muhammad Owais	BCS22022
Wasif Ali	BCS22024

Project Advisor

Dr. Salman Naseer

Department of Information Technology
University of the Punjab, Gujranwala Campus, Gujranwala, Pakistan
(2025)

IoT-Based Smart Home Automation System Using Arduino

Executive Summary

This Software Design Specification (SDS) outlines the technical design of the IoT-Based Smart Home Automation System developed using ESP32 microcontrollers, Firebase cloud services, and a Flutter mobile application. It converts the SRS requirements into detailed architectural components, module interactions, data flows, and hardware–software integration strategies.

The system design focuses on secure communication between the mobile app, Firebase Authentication, Firebase Realtime Database, and ESP32 nodes. It defines how the system enables remote ON/OFF control, real-time device monitoring, and sensor-driven automation using temperature, motion, light-level, and water-level sensors. The SDS includes architecture diagrams, component designs, sequence flows, DFDs, and state-machine models that describe the behaviour of each module.

This document serves as a technical blueprint for implementing a scalable, maintainable, and robust smart home automation solution.

Table of Contents

<i>System Design</i>	6
1. Design Considerations	6
1.1 Assumptions and Dependencies.....	6
1.2 Limitations.....	6
1.3 Risks	7
2. Requirements Traceability Matrix.....	7
3. Design Models	8
3.1 Architectural Design	12
3.1.1 UML Class Relationship Diagram	13
3.1.2 UML Component Diagram	14
3.1.3 Mapping Modules and Components to Architecture Style.....	15
3.2 Data Design.....	15
3.2.1 Data Dictionary (OO Approach)	16
3.3 User Interface Design	19
3.3.1 Screen Images.....	20
3.3.2 Screen Objects and Actions.....	22
3.4 Behavioural Model.....	23
3.4.1 Interaction Diagrams	23
3.4.2 State Diagrams	25
4. Design Decisions	27
4.1 Object-Oriented Design Pattern	27
4.2 Communication Handler Abstraction.....	27
4.3 Algorithmic Approach.....	28
4.4 Database Normalization	28
4.5 UI Design Strategy	28
4.6 Platform and Framework Decisions.....	28
5. Summary.....	28
<i>References</i>	29

List of Tables

Table 1: Requirements Traceability Matrix	7
Table 2: Data Dictionary for AutomationManager Class	16
Table 3: Data Dictionary for ESP32Controller Class	16
Table 4: Data Dictionary for Device Class	17
Table 5: Data Dictionary for Sensor Class	17
Table 6: Data Dictionary for User Class	18
Table 7: Data Dictionary for FirebaseService Class	18
Table 8: Data Dictionary for SubscriptionManager Class	19

List of Figures

Figure 1: Class Diagram	10
Figure 2: Interaction Diagram.....	11
Figure 3: State Transition Diagram.....	11
Figure 4: Architecture Diagram	12
Figure 5: UML Class Relationship Diagram	13
Figure 6: UML Component Diagram	14
Figure 7: Splash Screen	20
Figure 8: Login Screen.....	21
Figure 9: Register Screen.....	21
Figure 10: Dashboard Screen.....	22
Figure 16: Sequence Diagram - Device Control Interaction	24
Figure 17: Sequence Diagram - Login Process.....	24
Figure 18: Sequence Diagram – Automation Rule Execution.....	25
Figure 19: State Diagram - Automated Device Control	26
Figure 20: State Diagram - Smart Light Control	27

System Design

The system operates in a cloud-connected IoT architecture where a Flutter mobile application, Firebase services, and ESP32 microcontrollers work together to provide smart home automation. The mobile app communicates with Firebase Authentication for secure login and with Firebase Realtime Database for device commands, automation settings, and real-time status updates. ESP32 devices connect directly to the home Wi-Fi network and synchronize with Firebase to receive commands, upload sensor readings, and execute automation logic.

Each ESP32 node controls relays for appliances and reads data from sensors such as DHT11 (temperature), PIR (motion), LDR (light level), and water-level sensors. Automatic Mode is processed locally on the ESP32, ensuring fast response even when the mobile app is inactive. The design supports modular expansion, allowing additional ESP32 nodes or sensors to be added without major architectural changes.

1. Design Considerations

The system must maintain reliable communication between the mobile app, Firebase cloud, and ESP32 hardware. Since ESP32 devices have limited memory and processing power, the design must remain lightweight while still supporting real-time updates and automation. The architecture should be scalable, allowing multiple devices and sensors to be added easily. Security, low latency, and efficient data synchronization are key considerations.

1.1 Assumptions and Dependencies

- A stable internet connection is available for Firebase Authentication and Realtime Database synchronization.
- ESP32 devices can reliably connect to the home Wi-Fi network.
- Firebase services remain available for authentication and data storage.
- Users have compatible smartphones capable of running the Flutter mobile application.
- Appliances connected to ESP32 relay modules operate within safe voltage and current limits.
- Sensor modules (DHT11, PIR, LDR, water-level) function within expected accuracy ranges.

1.2 Limitations

- System performance depends on internet quality; slow or unstable networks may cause command delays.
- ESP32 has limited RAM and processing power, restricting extremely complex automation logic.
- Firebase free tier has usage limits that may affect large-scale deployments.

- Wi-Fi range limitations may affect ESP32 connectivity in large homes.
- Cloud latency may vary by region, affecting real-time responsiveness.

1.3 Risks

The system may face issues like network delays, hardware faults, or data sync problems, so the design includes fallback automation and simple recovery steps to keep it stable.

- **Network Failure:** Commands or status updates may be delayed or lost during internet outages.
- **Security Threats:** Unauthorized access attempts may target Firebase or device endpoints.
- **Hardware Failure:** Sensor or relay malfunction may disrupt automation behaviour.
- **Data Sync Issues:** Temporary desynchronization may occur if ESP32 loses connection.
- **Scalability Risk:** Excessive devices on a single ESP32 may overload its processing capacity.

2. Requirements Traceability Matrix

This matrix maps each functional requirement from the SRS to the corresponding design components, modules, and hardware elements defined in the System Design Specification (SDS). It ensures that every requirement is implemented and accounted for during design.

Table 1 shows the requirement traceability matrix.

Table 1: Requirements Traceability Matrix

Requirement ID	Requirement Description	Design Specification
FR-1	The system shall allow a new user to register using a valid email and password.	Component “Authentication Module”
FR-2	The system shall allow a registered user to log in securely using Firebase Authentication.	Component “Authentication Module”
FR-3	The system shall allow users to add ESP32-based devices through a guided onboarding process.	Component “Device Onboarding Module”
FR-4	The system shall allow the user to remotely turn ESP32-controlled appliances ON or OFF.	Component “Device Control Module”
FR-5	The system shall allow the user to enable or disable Automatic Mode.	Component “Automation Controller”

FR-6	The system shall automatically control the fan based on temperature thresholds..	Component “Temperature Automation Module”
FR-7	The system shall automatically control lights based on motion detection.	Component “Motion Automation Module”
FR-8	The system shall automatically open or close curtains based on ambient light level.	Component “Light-Level Curtain Automation Module”
FR-9	The system shall automatically control the water pump based on tank water level.	Component “Water-Level Pump Automation Module”
FR-10	The system shall activate a buzzer during predefined peak-time intervals.	Component “Peak-Time Alert Module”
FR-11	The system shall handle communication between the mobile app, Firebase, and ESP32 devices.	Component “Communication Handler”
FR-12	The system shall allow users to view and upgrade their subscription plan.	Component “Subscription Management Module”

3. Design Models

This section presents the object-oriented design models used to describe the structure and behaviour of the IoT-Based Smart Home Automation System. These models illustrate how system components interact, how data flows between them, and how the system responds to user actions and sensor events.

- **Class Diagram**

The class diagram represents the static structure of the smart home automation system by identifying the main classes, their attributes, methods, and relationships. It models the core entities such as **User**, **Device**, **Sensor**, **ESP32Controller**, and **AutomationManager**, along with their interactions. This diagram provides a clear blueprint for implementing the system’s object-oriented architecture in both the Flutter mobile application and the ESP32 firmware.

Key classes typically include:

- **User:** Stores authentication details and subscription status.
- **Device:** Represents an ESP32-controlled appliance with attributes such as deviceID, type, state, and thresholds.

- **Sensor:** Represents temperature, motion, light-level, or water-level sensors connected to ESP32.
- **ESP32Controller:** Handles relay control, sensor reading, and communication with Firebase.
- **AutomationManager:** Implements Automatic Mode logic based on sensor inputs and user-defined thresholds.
- **FirebaseService:** Manages authentication, database reads/writes, and real-time synchronization.
- **SubscriptionManager:** Handles subscription validation and access control for premium features.

Class Diagram of IoT-Based Smart Home Automation System

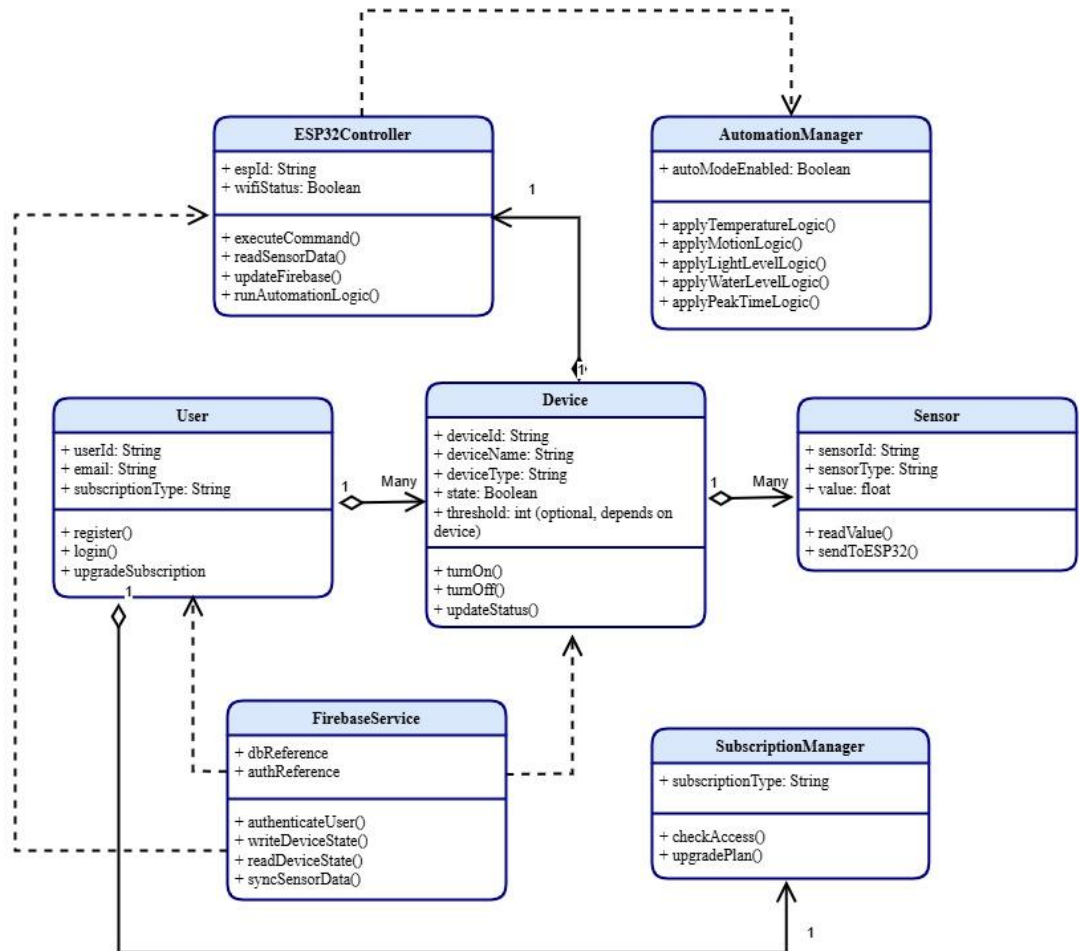


Figure 1 shows the class diagram.

Class Diagram of IoT-Based Smart Home Automation System

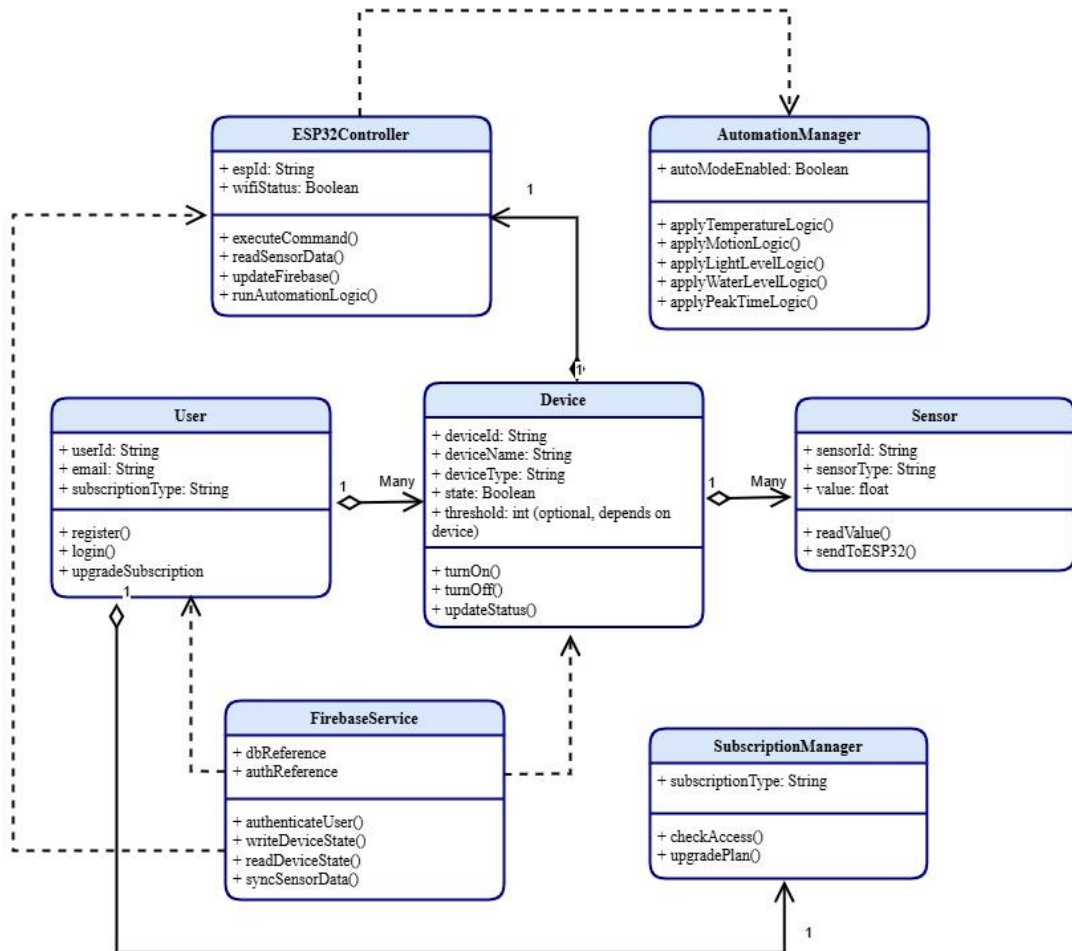


Figure 1: Class Diagram

- **Interaction Diagram (Sequence Diagram)**

The sequence diagram illustrates the interaction between the User, Mobile App, Firebase Service, ESP32 Controller, and Device when the user sends a command to turn a device ON. The User initiates the action through the Mobile App, which updates the device state in Firebase. The ESP32 Controller listens for this change, activates the corresponding relay on the Device, and updates the status back to Firebase. The Mobile App then reflects the updated state to the User in real time. **Figure 2** shows the UML Interaction Diagram.

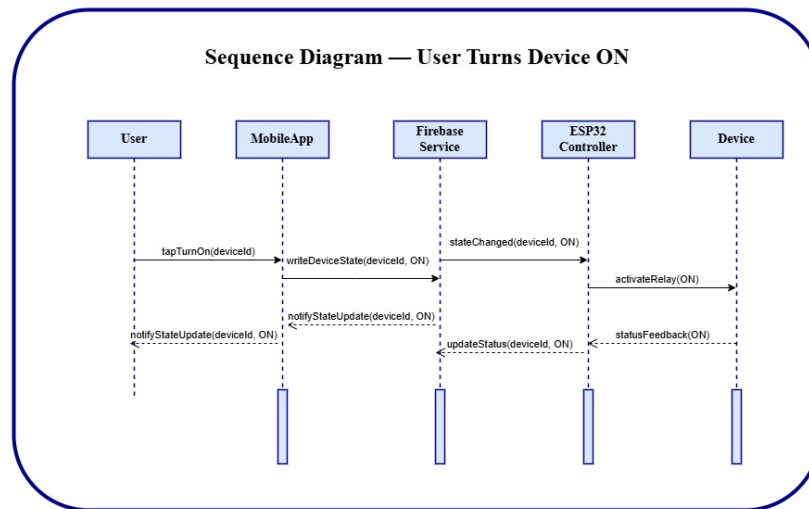


Figure 2: Interaction Diagram

- **State Transition Diagram**

The state transition diagram describes how a system component changes its internal state in response to specific events. It is particularly relevant for automation features such as temperature-based fan control, motion-triggered lighting, or peak-time water management. The diagram models the lifecycle of an automated device, showing how it transitions between states like OFF, ON, AUTO, and ALERT based on sensor inputs and automation logic. **Figure 3** shows the state transition diagram for automated device control.

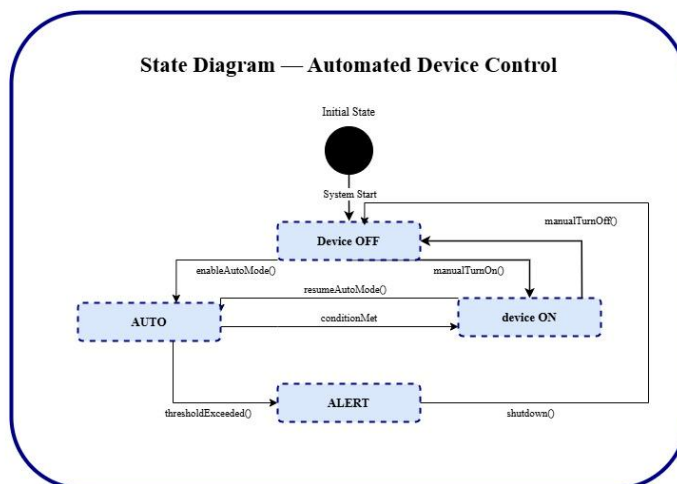


Figure 3: State Transition Diagram

3.1 Architectural Design

The proposed system follows a **three-tier client-server architecture**, ensuring clear separation of concerns, scalability, and maintainability. The **client tier** consists of the mobile application, which provides the user interface for device monitoring, manual control, and automation configuration. The **service tier** is implemented using Firebase services, which handle user authentication, real-time data synchronization, cloud storage, and communication between the mobile app and the controller. The **device tier** includes the ESP32-based controller, connected sensors, and relay modules that interact directly with the physical environment to execute automation rules and respond to user commands. The layered architecture and separation of concerns follow the recommended practices outlined in IEEE software design standards, ensuring maintainability and scalability (Society, 1998). **Figure 4** shows the smart home automation system architecture.

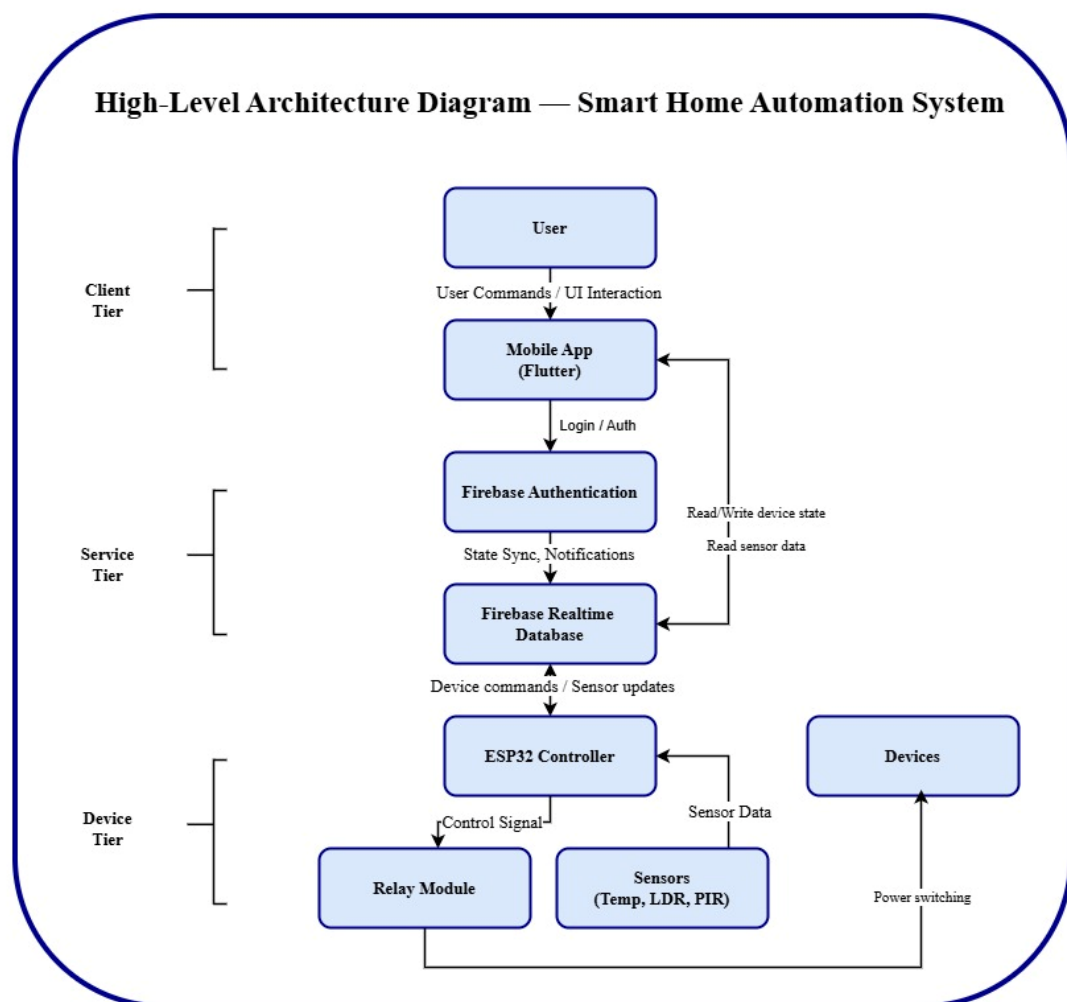


Figure 4: Architecture Diagram

3.1.1 UML Class Relationship Diagram

It outlines the core software structure of the smart home automation system, featuring key classes such as **User**, **Device**, **Sensor**, **FirebaseService**, **ESP32Controller**, and **AutomationManager**. It highlights essential relationships, including the one-to-many association between a Device and its Sensors, the Mobile App's dependency on Firebase for authentication and real-time updates, and the ESP32Controller's integration with sensors and relays for executing automation rules. **Figure 5** complements the architectural view by detailing how software entities collaborate to support authentication, device control, automation logic.

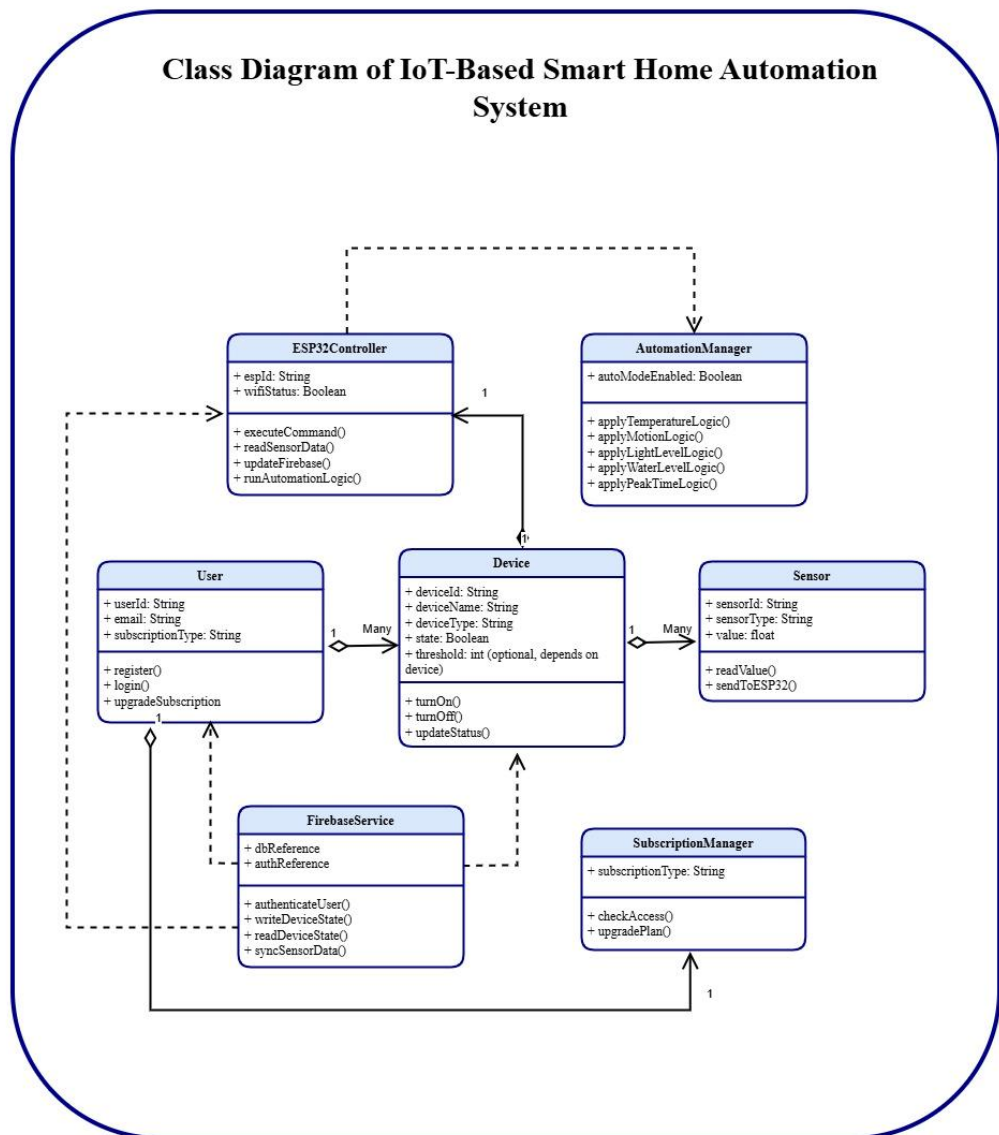


Figure 5: UML Class Relationship Diagram

3.1.2 UML Component Diagram

The UML Component Diagram illustrates the modular structure of the smart home automation system. It includes components such as the **Mobile App**, which interacts with **Firebase Authentication** and **Firebase Realtime Database** for user login, device control, and data synchronization. At the device layer, the **ESP32 Controller** communicates with the **Sensor Interface** and **Relay Control** components to process sensor data and execute automation logic. This diagram highlights clear separation of responsibilities across tiers and supports a scalable, maintainable system design. **Figure 6** highlights clear separation of responsibilities and supports a scalable, maintainable system design.

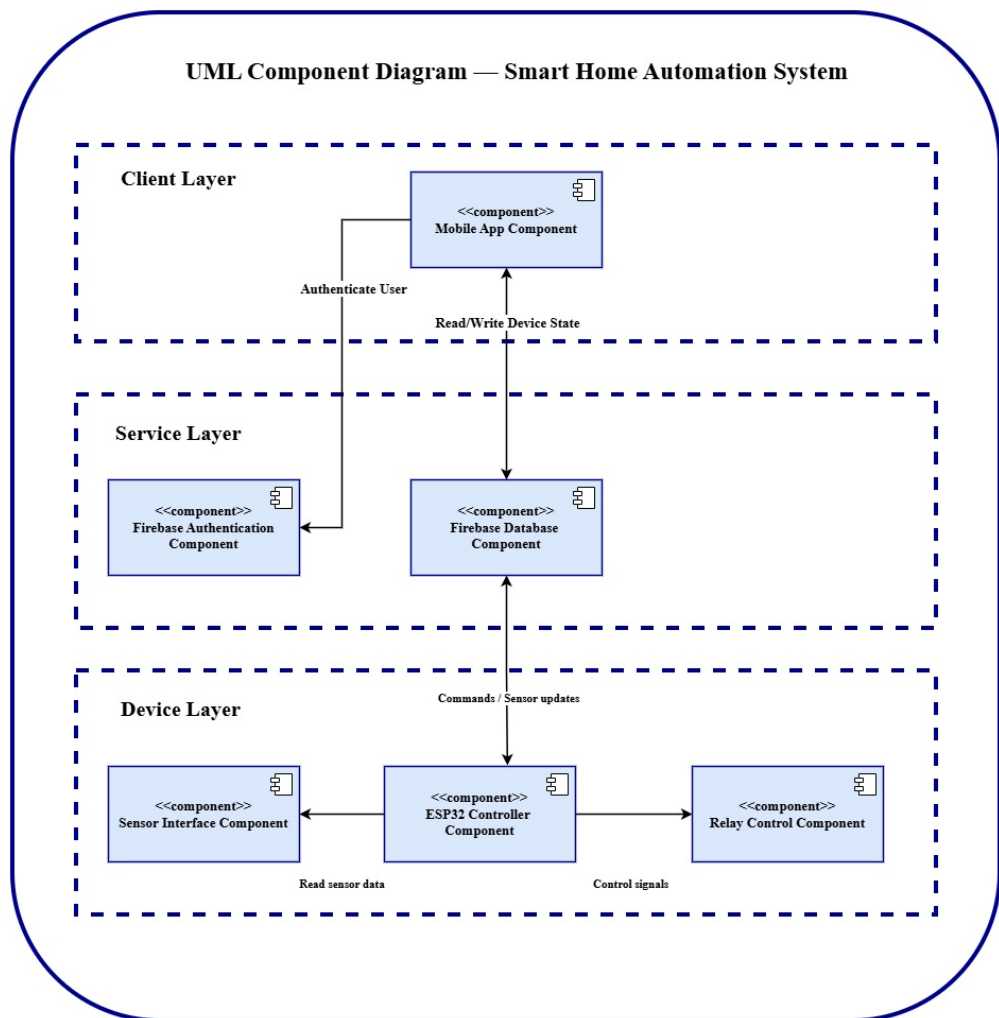


Figure 6: UML Component Diagram

3.1.3 Mapping Modules and Components to Architecture Style

The smart home automation system follows a **Layered Architecture** with a **Client–Server pattern**, ensuring separation of concerns, scalability, and maintainability. Each module is mapped to its respective layer based on its responsibility:

- **Presentation Layer (Client Side)**
 - **Mobile App Component**
Handles user interaction, authentication requests, device control commands, and real-time status display..
- **Service Layer (Server/Cloud Side)**
 - **Firebase Authentication Component**
Manages user login, registration, and secure access control.
 - **Firebase Database Component**
Stores device states, sensor readings, and synchronizes updates with the hardware layer.
- **Device Layer (Hardware Side)**
 - **ESP32 Controller Component**
Executes commands received from the cloud and coordinates device operations.
 - **Device Control Component (Relays)**
Performs physical switching of appliances.
 - **Sensor Interface Component**
Reads environmental data and sends updates to the controller.

This mapping reflects a multi-tier structure where the Mobile App acts as the client, Firebase services serve as the cloud backend, and the ESP32-based hardware layer enables real-world sensing and actuation.

3.2 Data Design

The data design defines how information in the smart home automation system is structured and managed. The system uses an object-oriented approach, where core entities such as users, devices, sensors, and the controller, are modelled with clear attributes and behaviours. All persistent data is stored in **Firebase Realtime Database**, organized as structured JSON to support fast lookups and real-time updates. Authentication and secure access control are handled through Firebase Authentication. This section also includes the data dictionary, which outlines each entity and its attributes. The database layout follows Firebase best practices to keep data consistent,

efficient, and easy to maintain. The Firebase Realtime Database structure was organized according to best practices suggested in the Firebase documentation, enabling efficient synchronization and reduced redundancy (Firebase, 2024).

3.2.1 Data Dictionary (OO Approach)

The data dictionary describes all major classes in the smart home automation system using an object-oriented approach. Each entry lists the class attributes and methods to clarify how data is represented and processed across the system.

3.2.1.1 AutomationManager (Class)

Table 2 shows the terms used in the class along with their description.

Table 2: Data Dictionary for AutomationManager Class

Term	Description
autoModeEnabled: Boolean	Indicates whether automation mode is active.
applyTemperatureLogic()	Applies temperature-based automation rules.
applyMotionLogic()	Applies motion-based automation rules.
applyLightLevelLogic()	Applies light-based automation rules.
applyWaterLevelLogic()	Applies water-level rules (if applicable).
applyPeakTimeLogic()	Applies time-based automation rules.

3.2.1.2 ESP32Controller (Class)

Table 3 shows the terms used in the class along with their description.

Table 3: Data Dictionary for ESP32Controller Class

Term	Description
espId: String	Unique identifier for the ESP32 board.
wifiStatus: Boolean	Indicates Wi-Fi connection status.
executeCommand()	Executes control commands received from Firebase.
readSensorData()	Reads data from connected sensors.
updateFirebase()	Sends device and sensor updates to Firebase.

runAutomationLogic()	Coordinates automation with AutomationManager.
----------------------	--

3.2.1.3 Device (Class)

Table 4 shows the terms used in the class along with their description.

Table 4: Data Dictionary for Device Class

Term	Description
deviceId: String	Unique identifier for the device.
deviceName: String	Human-readable name of the device.
deviceType: String	Type of device (e.g., light, fan).
state: Boolean	Current on/off status of the device.
turnOn()	Turns the device on.
turnOff()	Turns the device off.
updateStatus()	Updates the device status in the system.

3.2.1.4 Sensor (Class)

Table 5 shows the terms used in the class along with their description.

Table 5: Data Dictionary for Sensor Class

Term	Description
sensorId: String	Unique identifier for the sensor.
sensorType: String	Type of sensor (e.g., temperature, light).
value: Float	Latest reading from the sensor.
readValue()	Reads current sensor value.
sendToESP32()	Sends sensor data to the controller.

3.2.1.5 User (Class)

Table 6 shows the terms used in the class along with their description.

Table 6: Data Dictionary for User Class

Term	Description
userId: String	Unique identifier for the user.
email: String	User's email address.
subscriptionType: String	User's subscription level.
register()	Registers a new user.
login()	Authenticates an existing user.
upgradeSubscription()	Changes the user's subscription plan.

3.2.1.6 FirebaseService (Class)

Table 7 shows the terms used in the class along with their description.

Table 7: Data Dictionary for FirebaseService Class

Term	Description
dbReference: Map	Reference to device and sensor data.
authReference: Map	Reference to authentication data.
authenticateUser()	Verifies user credentials.
writeDeviceState()	Updates device state in Firebase.
readDeviceState()	Retrieves device state.
syncSensorData()	Stores sensor readings in Firebase.

3.2.1.7 SubscriptionManager (Class)

Table 8 shows the terms used in the class along with their description.

Table 8: Data Dictionary for SubscriptionManager Class

Term	Description
subscriptionType: String	Current subscription level.
checkAccess()	Validates whether a user can access a feature.
upgradePlan()	Updates the user's subscription.

3.3 User Interface Design

The user interface of the Smart Home Automation System is designed to be simple, intuitive, and responsive, allowing users to control devices and monitor sensor data with ease. The mobile app follows Material Design guidelines to ensure a clean layout, clear visual hierarchy, and consistent interaction patterns. Users can securely log in, view all connected devices, check their current status, and control them with a single tap. Real-time sensor readings such as temperature, humidity, and light levels are displayed clearly, and automation mode can be enabled or disabled at any time. The interface provides immediate visual feedback for every action through updated device states, refreshed sensor values, and helpful status indicators, ensuring a smooth and reliable user experience. The interface layout and visual hierarchy were designed in alignment with the official Material Design guidelines to ensure accessibility and consistency (Team, 2024).

3.3.1 Screen Images

Below are some of the screens showing our app display.

3.3.1.1 Splash Screen

This screen appears when the application launches. It displays the app's branding theme and transitions into the authentication flow. **Figure 7** shows the splash screen.

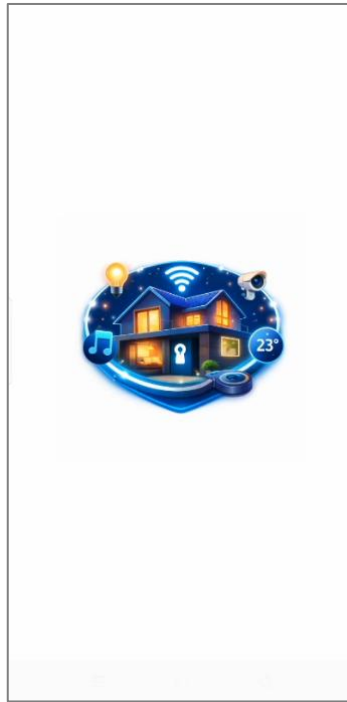


Figure 7: Splash Screen

3.3.1.2 Login Screen

This screen allows existing users to access the application by entering their email and password. It includes input validation, a login button, and a link to navigate to the registration screen. **Figure 8** shows the login screen.

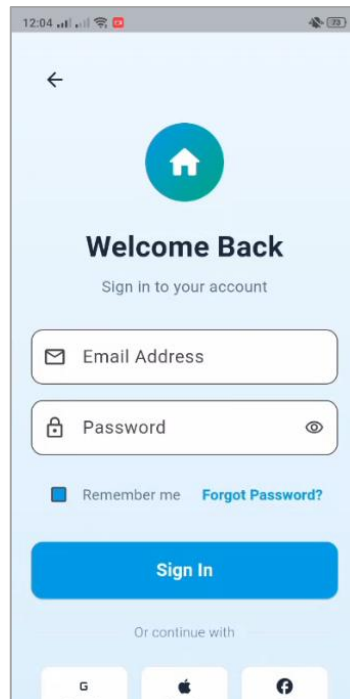


Figure 8: Login Screen

3.3.1.3 Register Screen

This screen enables new users to create an account by providing their name, email, and password. It includes a clean form layout, a create-account button, and a link to return to the login screen. **Figure 9** shows the register screen.

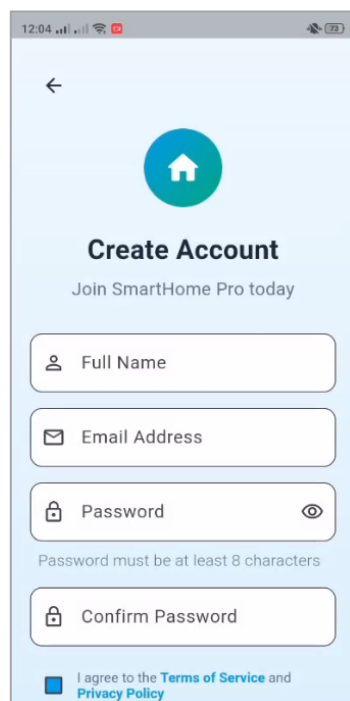


Figure 9: Register Screen

3.3.1.4 Dashboard Screen

The dashboard serves as the main navigation hub, allowing users to access devices, profile and overall system status. It uses large, accessible cards for quick interaction. **Figure 10** shows the dashboard screen.

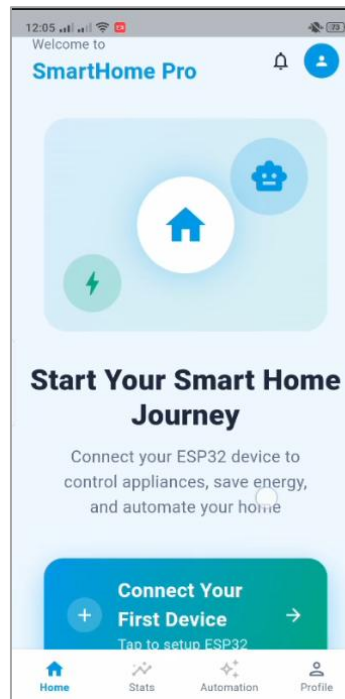


Figure 10: Dashboard Screen

3.3.2 Screen Objects and Actions

This section describes the key screen objects and the actions associated with them for two representative use cases.

- **Use Case 1: Turning a Device ON/OFF**
 - **Screen Objects**
 - **Device Card** – Displays device name, type, and current status
 - **Toggle Button** – Allows switching the device ON or OFF
 - **Status Indicator** – Shows real-time device state (e.g., green for ON, grey for OFF)
 - **Feedback Message** – Confirms success or failure of the action
 - **Actions**
 1. The user taps the **Toggle Button** on a device card.
 2. The system sends the command to update the device state.

3. The **Status Indicator** updates immediately.
 4. A **feedback message** appears, such as:
 - “Device turned ON successfully”
 - “Unable to update device state”
- **Use Case 2: Viewing Device Status**
 - **Screen Objects**
 - **Device Status Card** – Shows device name, icon, and ON/OFF state
 - **Sensor Value Display** – Shows any sensor values associated with that device (e.g., temperature for fan automation)
 - **Refresh Icon** – Allows manual refresh of device status
 - **History Button (optional)** – Opens past logs
 - **Actions**
 1. The user opens the Device Status Screen.
 2. The system retrieves the latest device state from Firebase.
 3. If the device is linked to a sensor, the sensor value is shown as part of the device status.
 4. The user may tap Refresh to update the status.
 5. The user may tap History to view past logs (if implemented).

3.4 Behavioural Model

The Behavioural Model describes how the Smart Home Automation System operates during runtime, focusing on the flow of interactions between the user, mobile app, Firebase services, and the ESP32 hardware layer. State and sequence diagrams illustrate how devices change state and how components communicate during key operations.

3.4.1 Interaction Diagrams

Interaction (Sequence) diagrams show how system components exchange messages during essential processes such as device control, login, and device synchronization.

3.4.1.1 Sequence Diagram - Device Control Interaction

Figure 11 illustrates how a user command flows through the system when controlling a device. The Mobile App sends a control request to **FirebaseService**, which updates the device state in the **Firebase Realtime Database**. The **ESP32Controller** listens for changes, executes the

command on the physical device through the relay module, and sends updated status back to Firebase, which the Mobile App then displays.

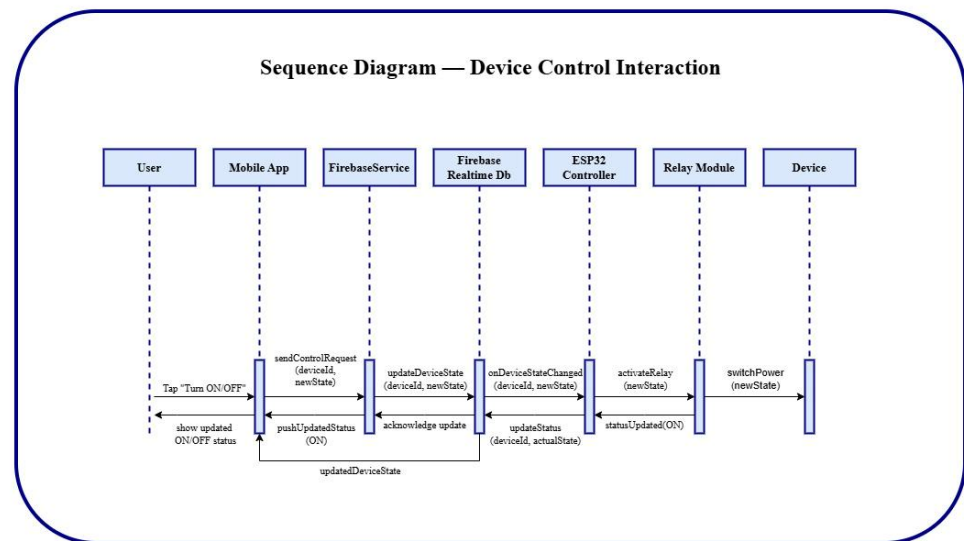


Figure 11: Sequence Diagram - Device Control Interaction

3.4.1.2 Sequence Diagram - Login Process

Figure 12 shows the authentication flow. The user enters credentials in the Mobile App, which sends a login request to **Firebase Authentication**. Firebase validates the credentials and returns the result. If successful, the Mobile App loads the user dashboard.

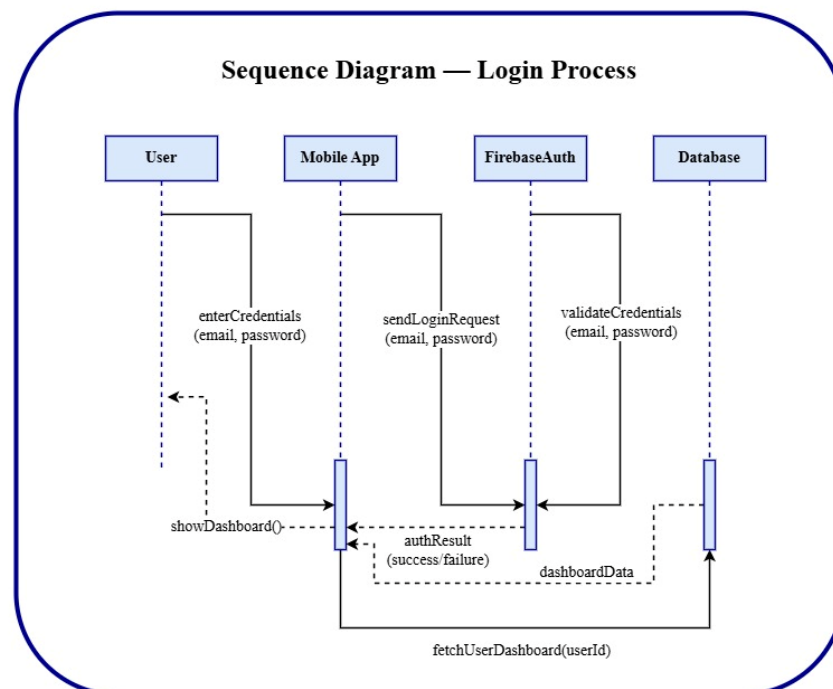


Figure 12: Sequence Diagram - Login Process

3.4.1.3 Sequence Diagram - Automation Rule Execution

Figure 13 shows how the system automatically reacts to sensor readings. When a sensor reports a new value, the Sensor Interface sends it to the ESP32Controller. The controller forwards the data to the AutomationManager, which checks the active automation rules. If a rule is triggered, the AutomationManager instructs the ESP32Controller to perform the required action, such as turning a device ON or OFF. The ESP32Controller activates the relay, updates the device state in Firebase, and the Mobile App receives the updated status in real time.

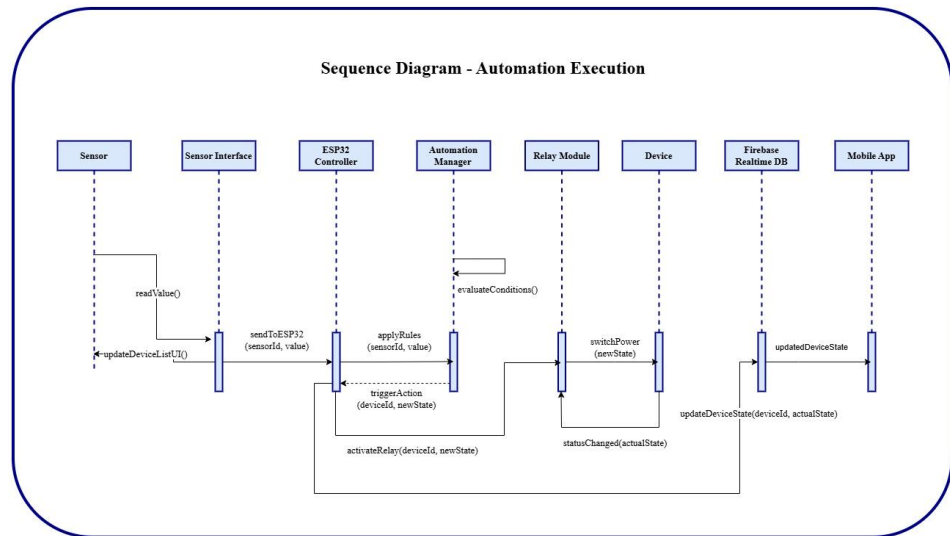


Figure 13: Sequence Diagram – Automation Rule Execution

3.4.2 State Diagrams

State diagrams illustrate how devices change their operational states in response to user actions, sensor inputs, and automation rules. They help clarify how the system behaves during runtime.

3.4.2.1 State Diagram 1: Automated Device Control

Figure 14 shows how an automated device (such as a fan or smart light) moves between states like Idle, Monitoring, Auto Mode, and ON/OFF. These transitions occur when new sensor readings arrive, automation rules are triggered, or the user manually overrides the device state.

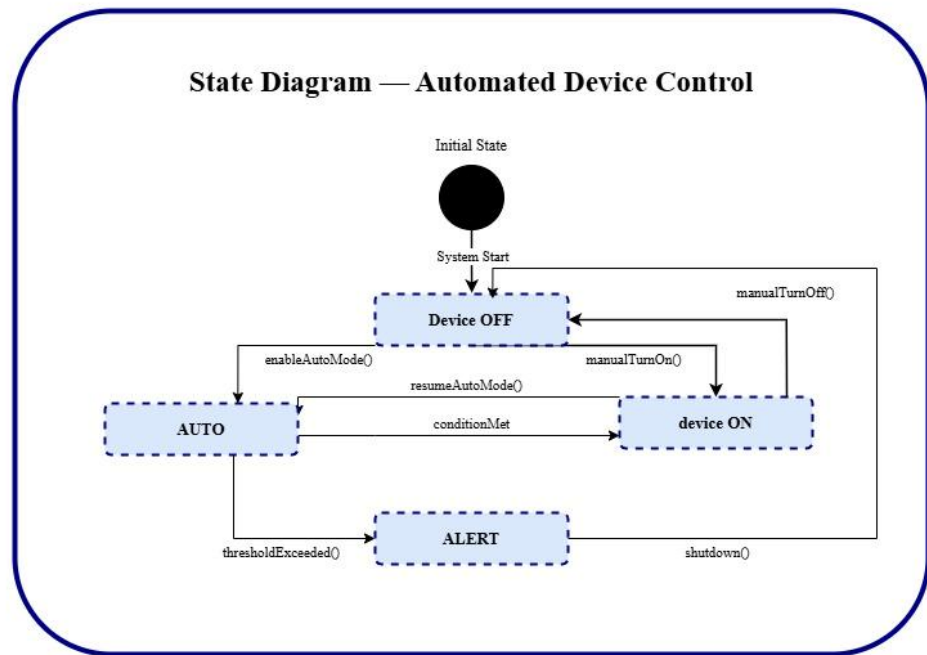


Figure 14: State Diagram - Automated Device Control

3.4.2.2 State Diagram 2 — Smart Light Control

Figure 15 illustrates how a smart light switches between OFF, ON, and AUTO states. In Auto Mode, the system adjusts the light automatically based on conditions such as motion or light level, while the user can override the state at any time.

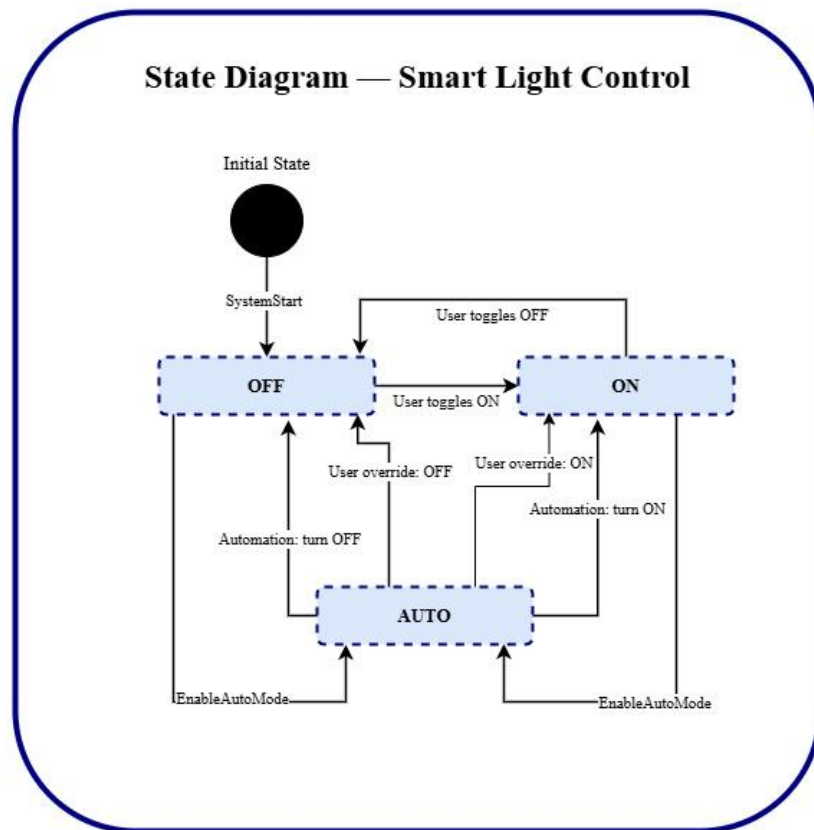


Figure 15: State Diagram - Smart Light Control

4. Design Decisions

This section highlights the key design choices made during the development of the Smart Home Automation System, covering architectural patterns, algorithms, database structure, and platform selection.

4.1 Object-Oriented Design Pattern

The system follows an Object-Oriented Design (OOD) approach to ensure modularity, scalability, and maintainability. Core components—such as **Device**, **User**, **FirebaseService**, **ESP32Controller**, and **AutomationManager**—are implemented as classes with clearly defined attributes and behaviours. This structure keeps responsibilities separated and makes it easy to introduce new device types or extend system features in the future.

4.2 Communication and Hardware Abstraction

Communication with the hardware is handled through the **ESP32Controller**, which abstracts Wi-Fi connectivity, sensor reading, and relay control. This separation keeps hardware-specific logic isolated from the mobile app and backend services. The ESP32 communication model is based on official Espressif documentation, ensuring reliable integration and easy upgrades if protocols or hardware modules change. The integration of ESP8266/ESP32 modules and their communication protocols was

implemented based on the specifications provided in the official Espressif technical documentation (Systems, 2024).

4.3 Algorithmic Approach

Automation logic uses a simple, threshold-based decision strategy. For example, a fan may turn ON when temperature exceeds a set limit. This approach is lightweight, predictable, and ideal for real-time embedded systems. More complex techniques—such as machine learning—were intentionally avoided because they are unnecessary for the project’s scope and would increase system complexity.

4.4 Database Normalization

The Firebase Realtime Database structure follows a 2nd Normal Form (2NF) layout to balance performance and data integrity. Device data is stored separately from user profiles, and redundant fields are minimized. This ensures fast read/write operations on mobile devices while keeping the data model clean and easy to maintain.

4.5 UI Design Strategy

The mobile app interface was designed using component-based prototyping in Figma. Material Design principles guided decisions around accessibility, spacing, and visual hierarchy. A card-based layout was chosen to present device controls clearly, and subtle colour accents were used to reinforce the smart-home theme. Iterative refinement ensured the screens remained consistent, readable, and SDS-ready.

4.6 Platform and Framework Decisions

- **Flutter** was selected for mobile development due to its cross-platform support and flexible UI capabilities.
- **ESP32 microcontrollers** were used for hardware control, offering stable Wi-Fi connectivity and seamless integration with Firebase.
- **Firebase Realtime Database** was chosen for its low-latency synchronization, ease of integration with Flutter, and suitability for real-time IoT applications.

5. Summary

In this chapter, the key design choices that shaped the Smart Home Automation System were outlined. These included adopting an object-oriented structure for modularity, using a dedicated hardware-abstraction layer to simplify communication with the ESP32, applying a threshold-based automation strategy for efficient real-time control, and organizing the Firebase database in 2NF to ensure fast and reliable access. Together, these decisions create a system that is scalable, maintainable, and responsive. Overall, the design choices directly support the project’s goals of reliable device control, transparent automation, and a smooth mobile user experience.

References

- Developers, G. (2024, Novemberx 10). *Flutter Documentation*. Retrieved from <https://docs.flutter.dev/>
- Firebase, G. (2024). *Firebase Documentation*. Retrieved November 10, 2024, from <https://firebase.google.com/docs>
- Society, I. C. (1998). *IEEE Recommended Practice for Software Requirements Specifications*. New York.
- Systems, E. (2024). *ESP8266/ESP32 Technical Reference*. Retrieved from <https://www.espressif.com/en/support/documents/technical-documents>
- Team, G. M. (2024). *Material Design Guidelines*. Retrieved from <https://m3.material.io>